

ĐỀ CƯƠNG CHI TIẾT KHÓA LUẬN TỐT NGHIỆP
HỌC KỲ I NĂM HỌC 2026 - 2027

Tên đề tài (viết chữ thường):

- Tên tiếng Việt: Nghiên cứu phương pháp kết hợp phân tích tĩnh và mô hình ngôn ngữ lớn nhằm nâng cao độ chính xác trong phát hiện lỗ hổng phần mềm
- Tên tiếng Anh: Investigating the combination of static analysis and large language models to improve software vulnerability detection accuracy

Cán bộ hướng dẫn: ThS. Tô Trọng Nghĩa

Thời gian thực hiện: Từ ngày 07/09/2026 đến ngày 26/12/2026

Sinh viên thực hiện:

Nguyễn Đức Trung – 23521678 – ATTT2023.2 - 0345265785

Nội dung đề tài (Mô tả chi tiết mục tiêu, phạm vi, đối tượng, phương pháp thực hiện, kết quả mong đợi của đề tài):

1. Bối cảnh đề tài

Lỗ hổng trong mã nguồn là một trong những nguyên nhân quan trọng gây ra các rủi ro an toàn phần mềm. Việc phát hiện và xử lý lỗ hổng từ sớm trong vòng đời phát triển có thể giúp giảm chi phí khắc phục và hạn chế nguy cơ bị khai thác khi hệ thống được triển khai.

Các công cụ phân tích tĩnh cho phép tự động kiểm tra mã nguồn mà không cần thực thi chương trình, nhờ đó phù hợp để tích hợp vào quy trình phát triển phần mềm. Tuy nhiên, hiệu quả của các công cụ này còn phụ thuộc vào tập luật và khả năng phân tích, đồng thời có thể phát sinh nhiều cảnh báo sai, làm tăng khối lượng kiểm tra thủ công.

Trong khi đó, mô hình ngôn ngữ lớn cho thấy tiềm năng trong việc hiểu mã nguồn và phân tích ngữ cảnh của các vấn đề bảo mật. Vì vậy, việc nghiên cứu phương pháp kết hợp phân tích tĩnh với mô hình ngôn ngữ lớn là một hướng tiếp cận phù hợp nhằm nâng cao độ chính xác, giảm cảnh báo sai và hỗ trợ phát hiện lỗ hổng phần mềm hiệu quả hơn.

2. Mục tiêu đề tài

Đề tài hướng đến xây dựng phương pháp cải thiện hiệu quả phát hiện lỗ hổng phần mềm trong mã nguồn thông qua việc kết hợp các công cụ phân tích tĩnh với mô hình ngôn ngữ lớn. Tận dụng khả năng tự động phát hiện các dấu hiệu bất thường trong mã nguồn của

công cụ phân tích tĩnh và khả năng phân tích ngữ nghĩa, ngữ cảnh của mô hình ngôn ngữ lớn [1].

Mục tiêu cụ thể:

- Nghiên cứu khả năng kết hợp kết quả từ nhiều công cụ phân tích tĩnh nhằm bổ sung thông tin và mở rộng khả năng phát hiện lỗ hổng trong mã nguồn.
- Xây dựng cơ chế chuẩn hóa và tổng hợp các cảnh báo do công cụ phân tích tĩnh sinh ra, làm cơ sở cho quá trình phân tích và xác minh.
- Sử dụng mô hình LLM chạy cục bộ để đánh giá lại từng cảnh báo dựa trên kết quả phân tích tĩnh và ngữ cảnh mã nguồn liên quan.
- Nâng cao độ chính xác của quá trình phát hiện lỗ hổng, đặc biệt là giảm số lượng cảnh báo sai mà vẫn hạn chế việc bỏ sót các lỗ hổng thực sự tồn tại [2].
- Sử dụng RAG để cung cấp thêm tri thức liên quan đến đặc điểm, nguyên nhân, điều kiện xuất hiện và biện pháp phòng tránh của lỗ hổng, giúp kết quả phân tích có căn cứ và phù hợp hơn với ngữ cảnh [3].
- Xây dựng một hệ thống thử nghiệm có khả năng thực hiện tự động các bước phân tích mã nguồn, xử lý cảnh báo, bổ sung tri thức, xác minh kết quả và tổng hợp báo cáo.
- Thực hiện đánh giá thực nghiệm trên các bộ dữ liệu có ground truth nhằm so sánh phương pháp đề xuất với việc chỉ sử dụng công cụ phân tích tĩnh hoặc kết hợp phân tích tĩnh với LLM nhưng không áp dụng RAG.

3. Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu phương pháp phát hiện lỗ hổng phần mềm trong mã nguồn bằng cách kết hợp kết quả của các công cụ phân tích tĩnh với khả năng phân tích ngữ nghĩa của mô hình ngôn ngữ lớn.

Phạm vi nghiên cứu bao gồm:

- Phân tích các lỗ hổng phần mềm có thể được phát hiện bằng công cụ phân tích tĩnh.
- Kết hợp kết quả từ các công cụ phân tích nhằm bổ sung thông tin và nâng cao phạm vi phát hiện.
- Chuẩn hóa, tổng hợp và xử lý các cảnh báo bảo mật để làm đầu vào cho mô hình LLM [4].
- Sử dụng một mô hình LLM chạy cục bộ để phân tích và xác minh các cảnh báo dựa trên mã nguồn và thông tin do công cụ SAST cung cấp.
- Áp dụng RAG như một kỹ thuật hỗ trợ, cung cấp thêm tri thức bảo mật có liên quan nhằm nâng cao chất lượng phân tích của mô hình [3].
- Đánh giá khả năng tăng độ chính xác, giảm cảnh báo sai và hạn chế bỏ sót lỗ hổng của phương pháp đề xuất.

Đề tài giới hạn trong phạm vi mã nguồn thuộc một ngôn ngữ lập trình chính và sử dụng các bộ dữ liệu có ground truth phù hợp để đánh giá.

4. Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài là phương pháp kết hợp phân tích tĩnh và mô hình ngôn ngữ lớn trong bài toán phát hiện lỗ hổng phần mềm [1].

Các đối tượng cụ thể:

- Mã nguồn phần mềm và các đặc trưng liên quan đến lỗ hổng bảo mật.
- Các cảnh báo được sinh ra từ các công cụ phân tích.
- Khả năng hỗ trợ giữa các công cụ phân tích tĩnh khi phát hiện lỗ hổng.
- Khả năng phân tích ngữ nghĩa mã nguồn và xác minh cảnh báo của mô hình LLM.
- Tri thức về lỗ hổng phần mềm được sử dụng để hỗ trợ quá trình phân tích của mô hình.
- Kỹ thuật truy xuất và bổ sung tri thức phù hợp với từng cảnh báo.
- Các phương pháp đánh giá độ chính xác, tỷ lệ cảnh báo sai và khả năng bảo toàn cảnh báo đúng.
- Các bộ dữ liệu mã nguồn phục vụ xây dựng và đánh giá thực nghiệm.

5. Phương pháp thực hiện

5.1. Khảo sát và phân tích

Khảo sát các kỹ thuật phân tích tĩnh, khả năng ứng dụng mô hình LLM trong phát hiện lỗ hổng và các phương pháp bổ sung tri thức cho mô hình. Trên cơ sở đó, xác định yêu cầu, kiến trúc và tiêu chí đánh giá của hệ thống.

5.2. Xây dựng quy trình phân tích tĩnh

Tích hợp các công cụ phân tích tĩnh để quét mã nguồn và phát hiện các vị trí có khả năng tồn tại lỗ hổng. Kết quả từ các công cụ được chuẩn hóa về một định dạng chung, sau đó được tổng hợp và xử lý để xác định những cảnh báo trùng lặp hoặc có liên quan.

5.3. Trích xuất ngữ cảnh phân tích

Đối với mỗi cảnh báo, hệ thống thu thập đoạn mã nguồn và các thông tin liên quan cần thiết. Việc lựa chọn ngữ cảnh phù hợp giúp mô hình tập trung vào vị trí có khả năng tồn tại lỗ hổng, hạn chế thông tin dư thừa và giảm chi phí xử lý [4].

5.4. Tích hợp mô hình LLM

Một mô hình LLM chạy cục bộ được sử dụng để phân tích và xác minh từng cảnh báo. Mô hình nhận đầu vào gồm kết quả phân tích tĩnh, ngữ cảnh mã nguồn và các thông tin bảo mật liên quan, sau đó đưa ra nhận định về tính hợp lệ của cảnh báo cùng với giải thích tương ứng [2].

5.5. Cải thiện mô hình LLM

Xây dựng cơ sở tri thức chứa các thông tin về đặc điểm, nguyên nhân, điều kiện xuất hiện và biện pháp phòng tránh lỗ hổng. Kỹ thuật RAG được sử dụng để truy xuất tri thức phù hợp với từng cảnh báo và bổ sung vào đầu vào của mô hình LLM [3].

5.6. Xây dựng hệ thống thử nghiệm

Hệ thống được xây dựng theo quy trình tổng quát:



5.7. Xây dựng thực nghiệm và đánh giá

Mô hình đề xuất được đánh giá trên ít nhất hai bộ dữ liệu, bao gồm dữ liệu kiểm thử có kiểm soát và dữ liệu từ các dự án phần mềm thực tế [5].

Các cấu hình so sánh dự kiến gồm:

- Chỉ sử dụng công cụ SAST.
- Kết hợp SAST và LLM.
- Kết hợp SAST, LLM và RAG.

Hiệu quả của từng cấu hình được đánh giá thông qua những chỉ số phổ biến như Precision, Recall và F1-score. Ngoài ra, đề tài xem xét số lượng cảnh báo sai được giảm, khả năng bảo toàn cảnh báo đúng, thời gian xử lý và tài nguyên cần sử dụng.

6. Kết quả mong đợi

- Xây dựng được quy trình tự động phân tích mã nguồn và thu thập cảnh báo từ nhiều công cụ phân tích tĩnh.
- Chuẩn hóa và tổng hợp được kết quả từ các công cụ thành dữ liệu thống nhất phục vụ quá trình phân tích.
- Tích hợp thành công mô hình LLM để xác minh các cảnh báo bảo mật.
- Xây dựng được cơ sở tri thức và thành phần truy xuất thông tin phù hợp với từng loại cảnh báo.
- Nâng cao độ chính xác của kết quả phát hiện lỗ hổng so với việc chỉ sử dụng công cụ phân tích.
- Giảm số lượng cảnh báo sai cần được kiểm tra thủ công, đồng thời hạn chế loại bỏ nhầm các cảnh báo đúng.
- Đánh giá mức độ đóng góp của LLM và kỹ thuật được áp dụng đối với hiệu quả chung của hệ thống.
- Chứng minh khả năng hoạt động của phương pháp trên nhiều bộ dữ liệu và trong môi trường thử nghiệm gần với dự án thực tế.
- Đưa ra các nhận xét về ưu điểm, hạn chế và khả năng mở rộng của phương pháp trong tương lai.

Kế hoạch thực hiện(Mô tả kế hoạch làm việc, thời gian biểu và phân công công việc cho từng sinh viên tham gia):

1. Kế hoạch làm việc

Giai đoạn 1: Khảo sát và xác định yêu cầu:

- Khảo sát các phương pháp phát hiện lỗ hổng bằng SAST và LLM.
- Nghiên cứu kỹ thuật RAG.
- Xác định phạm vi lỗ hổng, bộ dữ liệu và tiêu chí đánh giá.

- Hoàn thiện kiến trúc tổng thể và kế hoạch thực nghiệm.

Giai đoạn 2: Xây dựng thành phần SAST

- Tích hợp các công cụ SAST vào hệ thống.
- Xây dựng cơ chế thu thập và chuẩn hóa kết quả.
- Tổng hợp các cảnh báo trùng lặp hoặc có liên quan.
- Trích xuất mã nguồn và ngữ cảnh cần thiết cho từng cảnh báo.
- Kiểm thử quá trình quét trên các chương trình mẫu.

Giai đoạn 3: Xây dựng thành phần LLM

- Thiết lập môi trường chạy mô hình LLM.
- Xây dựng giao tiếp giữa hệ thống và mô hình.
- Thiết kế prompt và định dạng đầu ra.
- Xây dựng cơ chế phân tích từng cảnh báo.
- Kiểm tra tính hợp lệ của kết quả và bằng chứng do mô hình cung cấp.

Giai đoạn 4: Xây dựng cơ sở tri thức và RAG

- Thu thập và chuẩn hóa tri thức về lỗ hổng phần mềm.
- Xây dựng cấu trúc lưu trữ và đánh chỉ mục dữ liệu.
- Xây dựng cơ chế truy xuất tri thức phù hợp với từng cảnh báo.
- Tích hợp kết quả truy xuất vào đầu vào của mô hình.
- Đánh giá sơ bộ chất lượng truy xuất.

Giai đoạn 5: Xây dựng thực nghiệm

- Chuẩn bị các bộ dataset.
- Xây dựng các cấu hình đối chứng.
- Chạy công cụ SAST độc lập.
- Chạy cấu hình kết hợp SAST và LLM.
- Chạy cấu hình kết hợp SAST, LLM và RAG.
- Thu thập kết quả, thời gian xử lý và tài nguyên sử dụng.

Giai đoạn 6: Đánh giá và hoàn thiện

- Tính toán các chỉ số đánh giá.
- So sánh kết quả giữa các cấu hình.
- Phân tích các trường hợp xảy ra.
- Xác định ưu điểm, hạn chế và hướng phát triển.
- Hoàn thiện hệ thống, mã nguồn, báo cáo và tài liệu hướng dẫn.

Tài liệu tham khảo (theo chuẩn IEEE):

- [1] J. A. S. Ciavatta, J. R. B. Higuera, J. B. Higuera, J. A. S. Montalvo, T. S. Riera and J. P. Melero, "Integration of Large Language Models (LLMs) and Static Analysis for Improving the Efficacy of Security Vulnerability Detection in Source Code," *Computers, Materials & Continua*, vol. 86, no. 3, 2026.
- [2] J. Wagner, S. Müller, C. Näther, J.-P. Steghöfer and A. Both, "Towards Effective Complementary Security Analysis Using Large Language Models," in *2025 IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2025.

- [3] X. D. e. al, "Vul-RAG: Enhancing LLM-Based Vulnerability Detection via Knowledge-Level RAG," *ACM Transactions on Software Engineering and Methodology*, 2026.
- [4] A. Lekssays, H. Mouhcine, K. Tran, T. Yu and I. Khalil, "LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models," in *34th USENIX Security Symposium*, Seattle, WA, USA, 2025.
- [5] Z. Li, S. Dutta and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in 2025, *International Conference on Learning Representations*.

Xác nhận của CBHD (ghi rõ họ tên)	TP. HCM, ngày....thángnăm..... Sinh viên (ghi rõ họ tên)
---	---